UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

FIB

DYNAMIC 3D EROSION OF TERRAINS

LLUNA CLAVERA COMAS

Thesis supervisor
OSCAR ARGUDO MEDRANO (Department of Computer Science)

Degree
Master's Degree in Innovation and Research in Informatics (Computer Graphics and Virtual Reality)

Master's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Virtual terrains appear in a wide variety of applications: from entretainment to physical simulation or science visualization, a lot of virtual scenes are set on a digital landscape. Despite the ubiquity of virtual terrains on 3D scenes, the complex and fractal structure of natural landscapes still brings multiple challenges for the field of synthetic terrain generation.

Most virtual terrains are represented using 2D heightfields which, despite being highly convenient when generating, storing and manipulating terrain models, have a major drawback: heightfields can only represent 2.5D models. Because each point in a 2D plane has a set elevation and every point between the ground and the given elevation is considered as solid (and the terrain interior), we can not represent features like caves, arches or overhangs, where not all points between the ground and the maximum elevation are solid. Additionally, these representations of terrains also fail to reproduce complex geometry commonly found in rocky terrains, like sharp ridges, loose blocks or steep cliffs.

Many of these complex structures that can not be easily captured by 2D heightfields, arise from different types of weathering events, like percolation and freeze-thaw cycles. To address this limitations on virtual terrain models, ç our goal in this thesis is to study the mechanical weathering of rocky terrians to enhance models given from 2D heightfields and add the complex features that these fields fail to caputre.

Terrain enhancement through weathering simulation not only improves the realism and quality of the models, adding the aforementioned complex features, but it can also be applied dynamically to simulate the changes a terrain goes through over multiple years.

This project is built on a previous project on this same topic [1], using the same algorithm as a base, we worked on adressing some limitations of the previous project.

1.1 Contributions

As mentioned before, this project is heavily based on a previous one, and our proposed algorithm is built on the previous one. Despite that, we add mulitple improvements to the base algorithm to increase the quality of the results and its efficiency.

The main point we wanted to adress of the previous project was the lack of a physically based approach

to the algorithm, as the proposed algorithm made a lot of simplifications and ignored multiple physical interactions which made the method less accurate and realistic. However, during this thesis we also found other areas that could be improved and expanded to improve both the quality and the efficiency of the algorithm.

The limitations of the previous project and our imporvements will be discussed at length during these thesis, but our main contributions can be summarized in three large blocks: improvements to the decomposition of the terrain into cells, improvements in the efficiency of the algorithm and improvement on the quality of the algorithm by extending the simulation to a physically based one.

1.2 Set-up and tools

Another difference between the project we base this thesis on, albeit less significant, is the tools used. In the project that presented the algorithm we are extending, a blender pluging was implemented to show the method, however, here we have implemented our erosion model using C++ and we showed the results in a QT application using OpenGL for rendering.

To make the application, we reused the code provided by an article on terrain descriptors [2], as it already implemented many basic functionalities to work with virtual terrains. We then modified and expanded the application implementing our algorithm for mechanical weathering as well as implementing other functionalities to aid in the visualization and debugging of our method. Some of the changes that we implemented modify the basic functionalities of the program, so our project should not be seen as an extension to said application.

Finally, we used the C++ library Voro++ [3] to generate voronoi decompositions, a core aspect of our erosion algorithm.

Chapter 2

Preliminaries

To understand this project and our contributions to the field, we will first present some background information about the topic, so that the reader can better understand both the problem and our proposed solutions.

2.1 Terrains and Landscapes

T

2.1.1 General weathering effects

2.1.2 Freeze-thaw cycle

2.2 Terrain Representation and Modeling

Virtual terrains are a fundamental aspect of this thesis, so it is important to review current approaches to represent and work with these types of models. A deep dive into the state-of-the-art on terrain generation can be found on [4], but we are going to present only the most relevant methods for our project, as a full review is out of the scope of this thesis.

2.2.1 Terrain Representations

When representing terrains and landscapes there are multiple data structures and representations we can use depending on the ammount of information that we need to store and the nature of the model.

The main model representation techniques can be divided in *Elevation Models*, maninly focused on representing the surface of the terrain, and *Volumetric Models*, that aim to capture some interal properties of the terrain as well as more complex structures.

Elevation Models

Elevation models are the simplest ways of representing virtual terrains. They consist on assigning an height to every point on a 2D space. In a more formal way, a terrain is represented by a function $h : \mathbb{R}^2 \to \mathbb{R}$ that is at least C^0. From an height function we can easily extract the slope at one point $s(x, y)$ by using the norm of the gradient:

$$s(x,y) = \|\nabla h(x,y)\| = \left\|(\frac{\partial h}{\partial x}, \frac{\partial h}{\partial y})\right\| \tag{2.1}$$

And from the gradient (projected to $\mathbb{R}^3$) we can easily obtain the normal at one point $n(x,y)$:

$$n(x,y) = \frac{(-\frac{\partial h}{\partial x}, -\frac{\partial h}{\partial y}, 1)}{\left\|(-\frac{\partial h}{\partial x}, -\frac{\partial h}{\partial y}, 1)\right\|} \tag{2.2}$$

With the height, normal and slope we have all that we need for a basic representation of a terrain. Of course, as we are using a bivariate function to represent the terrain, we can only have one elevation for each point on the domain, and that makes features like caves or overhangs impossible to represent with this method.

Despite its limitations, elevation models are widely used for its simplicity and memory efficiency as they can capture the general structure of a landscape, given that complex features like overhangs often represent a small part of the overall terrain.

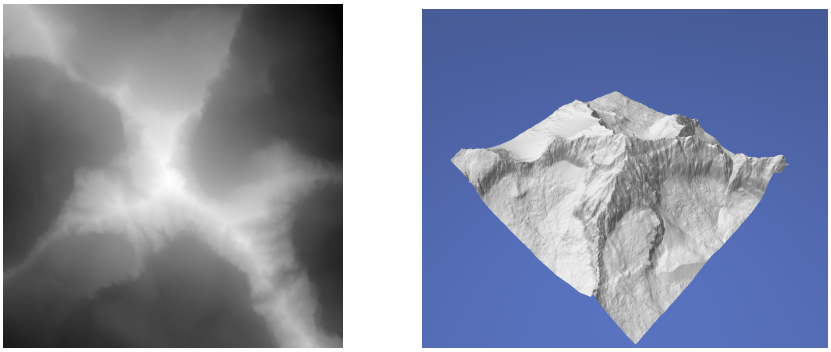
To expand on this topic, elevation models only require a bivariate function to represent the whole terrain, which can capture the whole terrain in a really simple closed-form expression, providing infinite precission with an extremly small ammount of memory. However, constructing functions that represent complex realistic terrains is highly challenging and, for that reason, elevation models commonly use *discrete heightfields* (also called *Digital Elevation Models* or DEM) to represent terrains.

Discrete Heightfields are a collection of altitudes in a 2D grid. In order to create a DEM, you have to divide the domain of the terrain (assuming a rectangular domain) into a grid of nxm cells, then, you assign an altitude to every cell of the grid. The grid will only contain concrete values for a set of discrete values, but you can obtain a height for every point of the domain by interpolating multiple values of the heightfield. This method limits the accuracy of the model (as we stop having infinite precission), but it has the advantadge of representing arbitrary terrains in a really simple way.

Heightfields are an extremly common way to represent terrains, and they can be easily stored in textures and monochromatic images (as figure 2.1 shows), sometimes making use of the texture interpolation capabilities on the GPU to accelerate computations on heightfields. Specifically, DEMs are used on [2], a project focused on terrain analysis that provides multiple metrics for terrains and whose code has been1 reutilized in this project to implement and test our erosion algorithm.

To finalize the topic of elevation models, they have been extended to represent some internal structure of the model by employing a layered represesentation of the terrain. Layered representations are basically an ordered collection of height functions, each function representing a layer of the model (which usually corresponds to a different material), resulting in a stack of elevations for each grid cell. A representation of these types of layered heightfields can be seen on figure 2.2

Layered representations have been used to simulate erosion ([5] and [6]) and, specially to simulate weathering events related to sediments as those types of rocks are naturally organized in layers. There are interesting works on this topics, but those will be covered on chapter 3



(a) Aneto heightfield (b) Aneto model on the app

Figure 2.1: Heightfield of the mountain Aneto as a monochromatic image and the terrain produced by said heightfield on our application

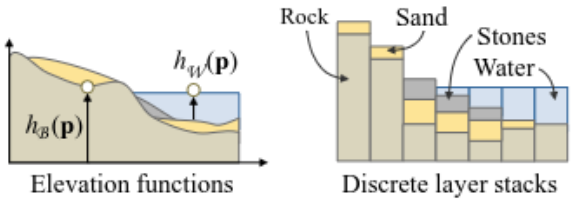


Figure 2.2: Illustration of stacked height functions, figure taken from [4]

Volumetric Models

As seen previously, layered representations are a way to represent different materials and properties of a terrains, but if we are really interested in encoding the internal structure of a landscape, we will have to use volumetric models. Volumetric models can be defined by a function $\mu : \mathbb{R}^3 \to \mathcal{M}$, where $\mathcal{M}$ denotes the set of materials used (air can count as a material). This function will be assigning every point of the 3D domain to a material, so now complex structures like overhangs can be represented, as well as different internal compositions of the terrain.

Despite volumetric models being defined with a function, in practice we will usually use a 3D grid with discrete samples and perform some interpolation to obtain values outside the fixed grid points, similarlly to what we did with DEMs. Each cell of the 3D grid is usually called a *voxel* and the transformation of a model into a set of voxels a *voxelization*. Voxelizations allow us to freely move and remove parts of the model, so they have potential to be used for algorithms that change the structure of the landscape, like for erosion simulation, and they have been explored in the literature [7], but they also have the huge drawback of being extremly expensive memory-wise. For that reason there has been a lot of work in the literature making voxel representations more efficient (like adaptaive grids) or presenting novel ways to represent models encoding volumetric information.

For our purposes, we could have used a voxelization to implement our algorithm, but we decided that decomposing the terrain in voronoi cells would produce more natural results in a more efficient manner. The main concepts of the decomposition that we have chosen will be explained on section 2.3.2.

Implicit Surfaces

From a field function $f : \mathbb{R}^3 -> \mathbb{R}$ over the domain Ω of the terrain you can implicity define a surface the following way:

$$\mathcal{S} = \{p \in \mathbb{R}^3 | f(p) = 0\} \tag{2.3}$$

That is, your sourface would be the set of all points which map to 0 in your scalar field. Going from a heightfield h to an implicit sourface is really straight-forward (although the opposite is not), you just need to define field f as $f(p) = h(p_{xy}) - p_z$, from that you can go to a usual volumetric representation, defining μ as $\mu(p) = 1$ if $f(p) < 0$ and $\mu(p) = 0$ otherwise (meaning the material will be rock if and only if the function f is negative) for example.

Scalar fields defined from elevation functions are a type of distance fields: for every point of the 3D space the field represents the vertical distance to the surface, being negative if the point is inside and positive otherwise. You can also use other distance fields to define surfaces (with any definition of distance that fits your situation) and you could easily transfom those distance fields into standard volumetric models by asigning different distances to different materials.

2.2.2 Terrain Synthesis

Terrain generation techniques can be divided in three main groups (although it is common to employ methods from the three groups in the same project):

- **Procedural Generation:** Procedural generation methods artificially generate terrains from scratch by applying randomized processes, like faulting or using noise.
- **Example-based:** Example-based methods create virtual terrains using scanned-data of real-world terrains.
- **Erosion Simulation:** Erosion simulations methods apply algorithms to simulate geomorphological processes when creating the landscape.

Synthetic terrains usually can not represent all the complexity of a natural landscape by procedural generation alone, and that is why erosion simiulations are so important, not only in order to "predict" a future for the landscape but also to add realism on virtual terrains. Even example-based terrains often fail to capture some aspects of the terrain they are trying to replicate, as scans have precission limitations (and the bigger the region you want to scan the more difficult it is to do it with accuracy), and erosion simulation can also intervene in those cases to add more complexity to the landscapes.

2.3 Model Decompositions

One of the core aspects of simluating erosion on 3D terrains is the model representation that we use. Any eroision simulation applied on a 3D model will modfiy the model and its properties, so proper data structures that allow for dynamic modifications of the model are key for these types of simulations. Additionally, applying erosion to an object also exposes the *interior* of said object, so any model representation that we use also has to encode some internal information of the model, as internal points of a model might become part of the surface at some point of the process.

Figure 2.3: Example of a tetrahedalization, taken from [9]

In this section we are going to explain some of the decomopositons that can be applied to virtual terrains in order to convert the full model in a set of independent parts that can move freely to modify the overall landscape. We already mentioned voxalizations before (on section 2.2.1), but due to voxelization limitations (mainly the memory cost), we explored more complex techniques to decompose our models.

2.3.1 Tetrahedralization

As you might already know, every polygon can be divided into a set of trtiangles that exactly match the perimeter of the polygon. Triangulizations can also be used to aproximate curved surfaces with arbitrary precission. In the case 3D polyhedra and volumes, tetrahedra are the equivalent of triangles, we can aproximate any volume using terahedra, with more efficiency than with any other polyhedra (like cubes).

We could use a tetrahedralization to decompose our terrain into tetrahedra and, then apply our algorithm over those pieces, and we could even further divide the tetrahedra into more tetrahedra to adjust the granularity of the decomposition. The concept of tetrahedralization and tetrahedral meshas has been used to simulate erosion in the past [8], but we have chosen a differnt decomposition that has some advantadges over tetrahedralizations and produce more irregular shapes, which better represents the shapes we can observe in nature.

2.3.2 Voronoi Decomposition

Given a set of sample points, a voronoi decomposition is a partition of the space into multiple regions, where every region is defined by the set of all points that are closer to a given sample than any other sample. In a more formal way, given a set of centroids $C = \{c_1, c_2 \dots c_n\}$, the set of regions defined by those centroids $\mathbf{R} = \{R_1, R_2, \dots R_n\}$ is defined as follows:

$$R_i = \{p \in \Omega | \forall c_j \in C, \;\; d(p, c_i) \leq d(p, c_j)\} \tag{2.4}$$

Where $\Omega \subseteq \mathbb{R}^3$ is the domain we are dealing with (In our case it will be the volume of the model we are subdividing), c_i is the centroid of region R_i and function d represents the euclidean distance. Notice that a point might lie in more than a region, when the distance from that point to multiple centroids is exactly the same. Points that lie on two regions create a plane defining the boundary between those two regions.

In this thesis we decided to use voronoi cells for multiple reasons. First, they can be generated with

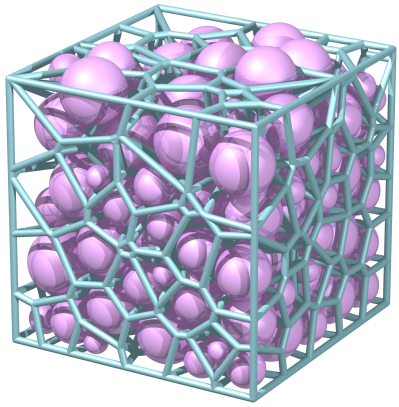


Figure 2.4: Example of a voronoi decomposition of a cube, showing the particles and faces of the decomposition. Figure taken from the documentation of the C++ library Voro++ [3]

a lot of flexibility (just chose an arbitrary set of points) and we can adapt the decomposition depending on our needs. Additionally, the generated cells are not very regular, with varying sizes and shapes, which produces a more natural rocky appearance than other more regular decompositions.

Chapter 3

Previous Work

There has been a multitude of proposed approaches to the problem of erosion simulation, each using different techniques and tackling different parts of the problem. To better understand our method and how it compares to previous solutions, we present a summary of the literature in the field, with their limitations and their differences with our approach. Additionally we will also discuss the paper we are basing our algorithm on [1], as our project can be seen as an extension to their work.

3.1 Work in Applied sciences

Simulation of Crack propagation using eigenerosion

3.2 Work in Computer Graphics

3.2.1 Related Methods in Computer Graphics

In this section we present work that is not focused on terrain erosion but it is still somewhat related to the topic.

Spring-Mass Systems and Brittle Fracture

Solid fracture is a fundamental aspect of simulating rock erosion, specially when simulating the fractures caused by the infiltration of water and its freezing. To simulate breakage one common and simple approach is to use spring-mass systems and remove overly extended springs. This approach was demonstrated years ago in [10] and [11].

A more recent approach in this topic was presented in [12], where they explore the use of peridynamic systems for brittle fracture. Peridynamic systems are spring-mass systems with two particular properties: they decouple stiffness from spring lenght and they also connect particles just by proximity instead of using 1-ring neighborhoods. An interesting part of the paper is that they have tried using Voronoi-Based Geometry to decompose the model and afpply their spring-mass algorithm. In our case we also use voronoi-cells to generate our *masses*, although the dynamics of the links and their fracture are different in our approach. Another approach that uses voronoi geometry for fracture can be found in [13], although in this case they use voronoi polygons over a surface instead of generating voronoi polyhedra in a volume.

Figure 3.1: A welsh dragon breaking. Taken from [12]

Figure 3.2: Example of wood dedcay of a cut tree trunk. Figure taken from [16]

Modeling Wood Decay

Modeling realistic trees have some parallels to modeling terrains: The goal is to represent a complex natural object that is being constantly modified through the interaction with its environment. Some work has been done [14] [15] on simulating tree decay, which would be the equivalent of simulating erosion on rocky terrains.

On this specific topic, the recent work of [16] (which is yet to be published on SIGGRAPH 2026) presents a really interesting method to simulate wood decay from fungal interaction. The method consists on dividing the tree model into *segments* and then simulating a fungus infiltrating the host and generating fractures in the tree segments. A fungal infiltration also creates conditions facilitating further decay. This is similar to the problem we are tackling: We divide a terrain into different components and then we simulate the water infiltrating the rock, fracturing it and creating conditions where more water infiltration is favoured. The specific mechanics of rock fracture and wood decay are very different, so techniques for wood decay are not really applicable to our work, but the topic has interesting parallels.

3.2.2 Weathering in Computer Graphics

Here we will show a summary of methods for erosion simulation in the literature.

Figure 3.3: Arch produced by wind erosion. Figure taken from [18]

Rock Erosion Simulation with Volumetric Models

This section of the literature is the colsest to our line of work, as we are approaching the problem using a decomposition of the terrain into discrete cells that can be freely moved. This specific approach has been tried multiple times before, although with different considerations.

For example, on [17] a similar approach to ours is presented: a rocky object is decomposed into multiple voxels connected by joints and then a physical simulation is run to determine the forces that each block and joint has to whistand to determine which blocks should be removed. In our case we also use cells and joints to represent the terrain, but our project additionally deals with the effect of water infiltrating the rock and destroying the joints.

Another example, and a highly recent one, was presented by [18]. The goal of the paper was to simulate sandstone dynamics with multiple effects like wind and fluvial erosion. The presented method represents a sandstone structure using a set of particles and then they modify the model using the following equation for each particle:

$$\frac{\partial b}{\partial t} = -\varepsilon(\sigma_c)(\mathcal{W} + \mathcal{F}) \tag{3.1}$$

Where b represents the *viability* which describes the expectancy that the rock has not been eroded, ε represents the *erodibility*, σ_c is the Cauchy stress tensor, and $\mathcal{W} and \mathcal{F}$ represent the wind and fluvial local erosion effects.

The problem solved by this simulator is similar to the one that we are tackling, but we are dealing with erosion by freezing and not with wind and fluvial erosion so our proposed solution will tackle other situations not covered by this method.

Terrain Amplification using Implicit Representations

The previous approaches use some sort of volumetric representation of the model (or at least attempt a volumetric decomposition) in order to modify individual components of the terrain, but other representations can also be used.

Specifically, the method presented in [19] is based on converting heighfields into implicit surface and then working on those implicit surfaces to generate complex 3D features such as arches or caves. In order to achieve this they also present a geological model to simulate different natural phenoma such as erosion and produce realistic results.

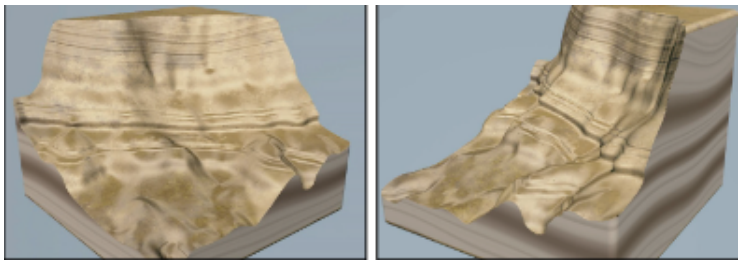


Figure 3.4: Example of erosion using implicit functions, the resulting terrain is smooth due to the nature of implicit functions.Taken from [19]

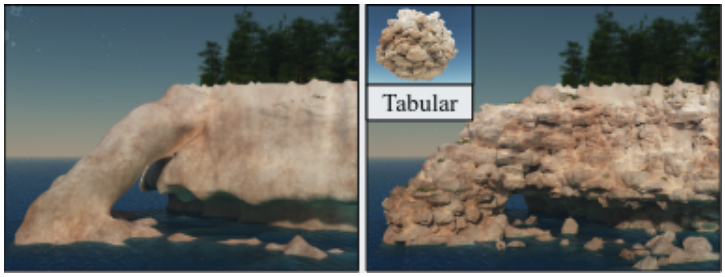


Figure 3.5: Terrain enhanced by generating fractured blocks according to the geology of the terrain. Taken from [20]

In order to represent the aforementioned features, the implicit funciton for the terrain has to be modified, and the resulting terrain will also be given as an implicit function. Implicit functions are really good at capturing smooth features, but representing sharp features such as vertical walls of rocky cliffs is a challenging task. For this reason, this method fails at capturing rough features generated by the erosion process, and this is a limitation that we addressed in our project by using voronoi cells instead of an implicit representation.

Another interesting work in the field is the one presented by [20]. In the article they present a method to enhance smooth terrains by adding realistical reproductions of rock fractures. The approch taken in the article consists on dividing the original terrain into blocks, whose shape and size is based on the properties of the given rock, and then combining the blocks with some parts of the terrain to obtain a volumetric representation of the fractured bedrock patterns. In this method, the resulting combined model is given as an implicit surface that combines the original terrain with the fractured blocks and, to achieve that, they proposed and applied an operator to reproduce the features of the fractured rock on the implicit surface.

To finish this section on implicit surfaces, the approach presented on [21] has some smilitudes with our method. PRESNET KRAST NETWORKS.

Erosion using Layered Representations

In section 2.2.1 we briefly discussed layered heightfield represesentations andf how they can be used for erosion simulatio, specially to simulate sediments and their interactions. Using this strategy, the work of [6] presents an interesting approach, that specially focus in the interaction between vegetation and erosion and their mutual interaction. In the paper, they divided the landscape in bedrock (unmoving), granular materials that may produce landslides and vegetation (dead and alive) and with that layered

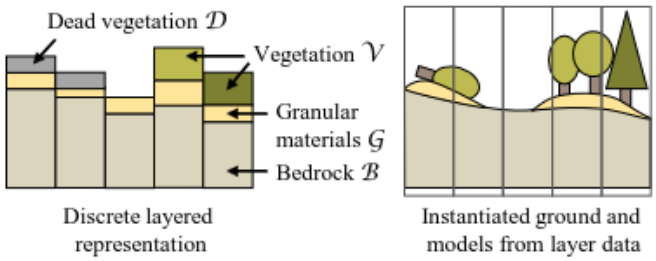


Figure 3.6: Illustration of stacked height functions, used to simulate interactions between vegetation and erosion. figure taken from [6]

representation they could run a simulation to modify those layers, representing the effect of erosion in the ecosystem, improving realism and showing the temporal evolution of a landscape. The layered representation they used can be appreciated on figure 3.6.

Another approach that uses layered representations is the one presented by [22] which tries to simulate the movement and interaction of tectonic plates to generate mountains with commplex folded strata. In the paper, a layered data structure was presented to represent the different tectonic plates and to compute the results of their colisions.

In our project layered representations are not suitable for our goal, as the main erosion mechansim that we study is the one caused by water entering the pores of the rock and breaking them from inside, causing fractures and potentially separating chuncks of rocks from the main structure. Layered representations can represent different features of the landscape with much more memory efficiency than other approaches and can also be used for erosion simulations, but they have limitations that make them not appropiate for certain types of erosion simulation, specially those that consider damage done over all the terrain and not just the interaction of different layers.

Machine Learning Approach

In the field of terrain modeling there have also been approaches that use machine and deep learning to improve the realism of the landscape and add complex details. In these types of methods, the goal is usually to add details to the terrain using machine learning models that are trained on real terrains or other suitable synthetic terrains. As other example terrains are used to train the models, erosion features can implicitly (and sometimes explicitly) be generated. These methods are a little bit far from our proposed solution but, despite that, there are interesting techniques using this approach.

In [23] they use a Conditional Generative Adversarial Network trained on real-world terrain to add realistic detail to a rough sketch provided by the user, moreover, they also have a framework to simulate the evolution of the terrain through erosion (with another adversarial network); [24] presents an approach that procedually enhances terrains by adding erosion patterns using structured noise; and [25] uses Fully Convolutional Neural Networks to upscale low-resolution terrains into plausible high-resolution ones.

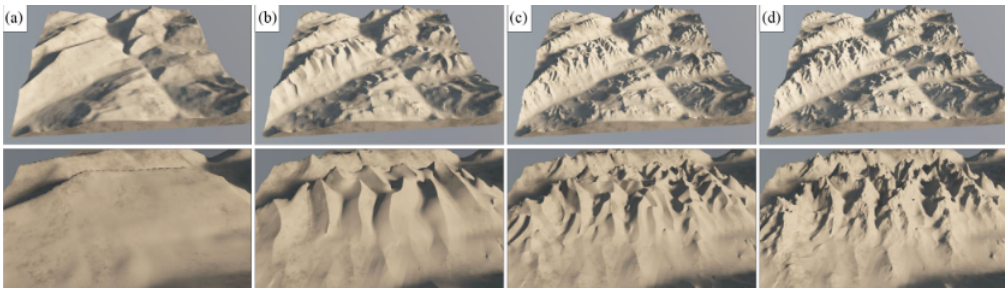


Figure 3.7: Example of terrain enhancenment through adding erosion patterns. Taken from [24]

3.3 Base Simulator for this Project

As mentioned on the introduction of this report (1), this thesis is built upon the work of [1], so we started from the algorithm they proposed and made some extensions and improvements. For this reason, in order to understand our projet we first have to understand the method presented in the previous project.

In the original paper [1] they present an approach to simulate terrain erosion that can be divided into two main parts: first they divide the terrain into independent cells conected by *links* between neighboring pieces, and, then, they apply an erosion algorithm to degrade the links and, eventually, remove the parts that have been disconnected from the terrain. Figure 3.8 shows the erosion simulation in action, with water infiltrating a terrain that has been decomposed into voronoi cells.

To perform the decomposition of the terrain into cells they sampled multiple points of the model and then generated a voronoi tessellation. In our project we still use the voronoi tessellation to generate the cell graph, but we modified the sampling to obtain better results. The original method used boolean operations between the voronoi tessellation and the original model so that the tessellation would fit the base terrain, but we tried to improve the decomposition so it could better fit the terrain without extra operations.

Regarding the algorithm that simulates water infiltration and link breakage, we implemented a mechanical model to represent link stress and modify those stress values once a link has been broken. This extension of the algorithm now modifies the state of the system after a fracture, instead of just modifying the state of one link, which can produce an effect called crack propagation. The addition of this mechanical model makes the method more based in natural and physical processes and it improves its realism.

Additionally, we also experimented with multi-resolution decompositions in order to improve the efficiency of the simulation without compromising too much on its quality. The algorithm can be applied in a less granular decomposition and then we can modify the more granular one according to the changes on the simplified level.

Furthermore, this thesis also changes a lot the used tools and the testing framework. The original project implemented a blender plugin to demonstrate their solution, while we implemented the algorithm into a C++ OpenGL application. Moreover, we have also changed the models we use for our application: On the original method, arbitrary models imported on blender were used, while we decided to focus on just DEMs, which are a pretty standard representation for terrains in computer graphics.

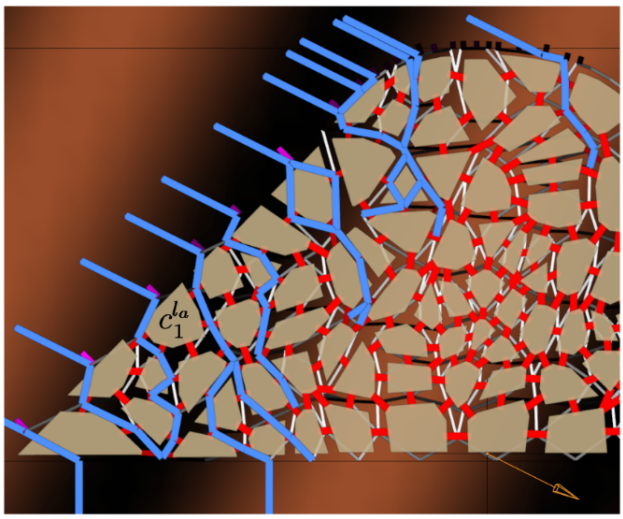


Figure 3.8: Example of water infiltrating a terrain divided into a set of voronoi cells connected by links. Taken from [26]

Finally, even if the basic algortihm is taken from another project, we are going to explain in full detail all parts of the algorithm so that the reader can understand our implementation just by reading this thesis. We are going to highlight our main contributions, but keep in mind some smaller details of the implementation may also be different to the original implementation (and we might not mention the change), as we took the main ideas of the method and implemented them in our own way.

Chapter 4

Model Decomposition

In order to represent the internal structure of a model, we decomposed the base model into multiple cells covering the whole volume of the object. A core part of this project has been focused on the model decomposition, as well as on algorithms to produce and improve said decomposition. In this section of the thesis we are going to take a deep dive into the basic method taken from [1] and all the improvements we added.

4.1 Base Method Overview

As statet eariler, the main idea of the decomposition process is to divide the original model into multiple shards that are connected with bonds (that we call links). In order to generate the fragments of the models, which will be called cells from now on, a voronoi tessellation is used, as they produce a more natural rocky appearance than other possible decompositions like voxelizations or tetrahedalizations. The process of generating is further divided in multiple steps: **Point Extraction**, **Voronoi Cells Generation** and **Links Generation**. Additionally we also experimented with multi-resolution decompositions, where one cell on the lower resolution contains multiple cells on the higher resolution level, this adds an additional step that consists on creating the lower resolution levels from the higher one.

In the original project, they divided the decomposition process in different steps, as it can be seen on 4.1, due to some changes in the method we changed a the steps a little. Despite that, the main outline of the process is similar and our changes reside in the specific steps.

4.2 Point and Cell Generation

Remember that voronoi tessellations are partitions of a model into different regions, where, given a set of centroids $C = \{c_1, c_2, \ldots, c_n\}$, the set of regions $\mathbf{R} = \{R_1, R_2, \ldots, R_n\}$ is defined by:

$$R_i = \{p \in \Omega | \forall c_j \in C, \;\; d(p, c_i) \leq d(p, c_j)\} \tag{4.1}$$

Where $\Omega \in \mathbb{R}^3$ is the domain of our model, d represents the euclidean distance and c_i is the centroid of region i.

This definition implies that the only thing we need in order to construct a voronoi tessellation is a set of centroids that will define our regions, then the regions will become our cells defined by the planes

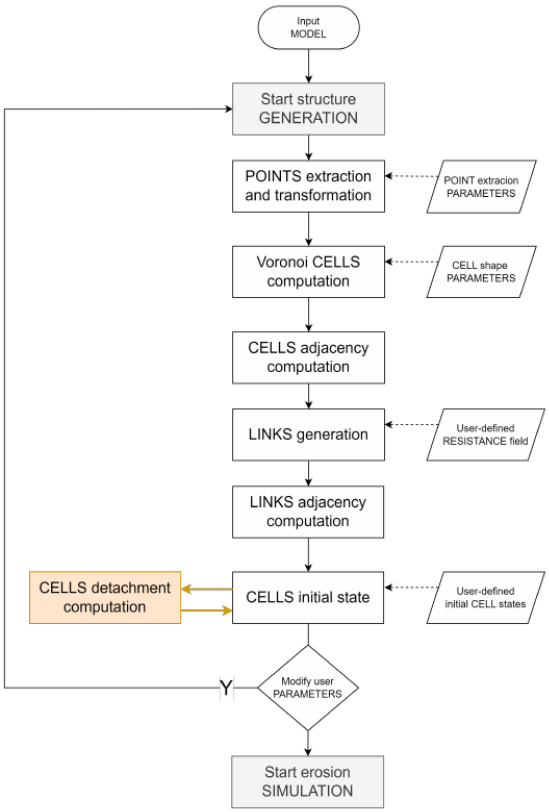


Figure 4.1: Diagram of the original method. Taken from [26]

that form their boundaries. The number of centroids chosen and their positions heavily affect the result of the tessellation, so the sampling method that we decide to implement will be highly relevant for the end result. In addition to the sampling, one more thing that we will need to take into account is the fact that the regions that lie on the boundary of the region will be technically unbounded and we will have to manually add walls to cut them so they do not leave the boundary of the original model.

4.2.1 Cell Generation

The first step of the voronoi tessellation would be to sample the points that we will use as centroids of our cells. Despite that, we think it is better to first explain the process of constructing the cells from the points, as we took it into account when designing our sampling.

In order to construct the cells from the centroids we offload the computation to the library Voro++ [3], which given a set of points and a boundary (with restrictions on the boundary that will be discussed in the future) gives us a set of polyhedra representing the different voronoi cells. Even if this part of the computation is offloaded to another library it is important to know about the process of construction of one voronoi cell.

Given a pair of centroids c_i and c_j, the whole region is divided into two half-spaces: H_{ij}, where points are closer to c_i than c_j; and H_{ji}, for points closer to c_j. More formally we can define H_{ij} the following way:

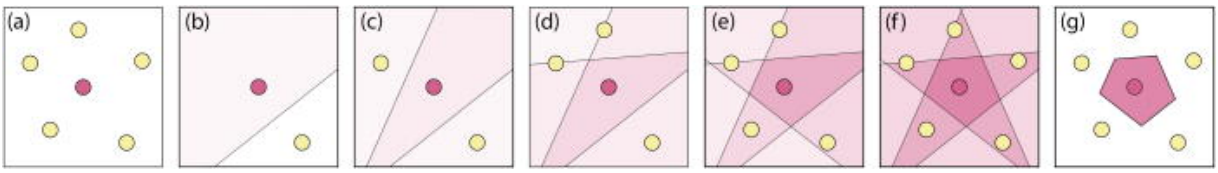


Figure 4.2: Process of construction of a voronoi cell from the intersection of half-spaces. figures (b) to (f) show the process of cutting the space into multiple half-spaces, one half-space at a time. Figure taken from [27]

$$H_{ij} = \{p \in \Omega | d(p, c_i) \leq d(p, c_j)\} \tag{4.2}$$

Every point in R_i is closer to c_i than any other centroid c_k, so it follows that it also has to be in H_{ik} for every $k \neq i$. Conversly, if we have a point $p \in \Omega$ in H_{ik} for every $k \neq i$, it follows that $p \in R_i$. This fact allows us to define the voronoi cell R_i as the intersection of a set of hall-spaces:

$$R_i = \bigcap_{j=1}^{n} H_{ij} \tag{4.3}$$

Each half-space is defined by the plane that is equidistant from both centroids, so this converts the problem of creating a voronoi cell into computing the intersection of multiple planes. Keep in mind that H_{ii} would define all the domain, since every point is at the same distance from c_i and c_i, so it does not have any effect adding it to the intersection. A nice visual demonstration of the construction of a voronoi cell from the intersection of half-spaces is shown in figure 4.2

Before ending the explanation about constructing cells, it is important to keep in mind that a pair of centroids define a plane where every point on the plane is equidistant from both centroids. Let us define this plane more formally, as we are going to use it later:

$$P_{ij} = P_{ji} = \{p \in \Omega | d(p, c_i) = d(p, c_j)\} \tag{4.4}$$

Neighboring Information

To run the erosion simulation, we will additionally need to create [links] between neighboring cells, which represent the bonds between different rocks. Adjacency information is also given by the Voro++ [3] library, but we wll provide an overview of the method to compute this information.

We have just commented that in order to generate a voronoi cell you will need to perform an intersection of halfspaces, which would imply incrementally intersecting your partial solution by one plane at a time. Each time you cut your partial solution by a plane P_{ij}, you can store the ID of the particle that j that generated said plane. At the end, just a few planes will remain as faces of the voronoi cell, and those planes will have an ID that indicates which other cell is sharing that specific face. Using this method we will compute the neighboring information while we compute the voronoi cells and we can use this information later to generate the links between cells.

Cell Attributes

To end this section about voronoi cells, it is important to hightlight what attributes we will also store for each cell, besides the vertices and edges that define its shape and its adjacency information.

- **Shape Information:** For each cell we will store its vertices, whit their position, and its faces, with the vertices that conform said face.
- **Additional Face Information:** For each face we will also keep its normal, its area, and the ID of the cell that is on the other side.
- **Cell State:** We define 3 possible states for a cell: **Solid**, which implies the cell belongs on the terrain (and it has not been removed yet by the erosion simulation); **Air**, which implies that the cell does not belong to the model; and **Core**, which makes the cell fixed and imposible to remove. A cell can not have two different states simultaniously.
- **Volume:** For some computations we are also interested in keeping track of the volume of the cell.
- **Boundary Information:** Finally, we will also need to keep track of the solid cells that are on the surface of the model, as they will be in contact with the air and the water infiltration algorithm will start by one of those cells.

4.2.2 Point Sampling

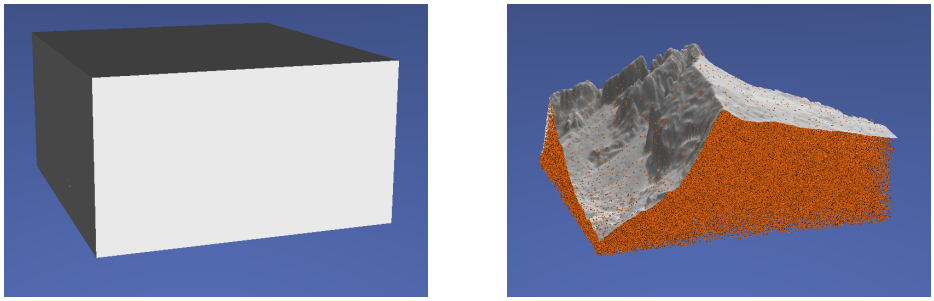
The first part of the method will be to sample multiple points inside the model to construct a voronoi decomposition. In this thesis, and unlike in the base method presented by [26], we are using DEMs to represent and load our terrains, so we are going to make use of that.

A heightfield $\mathcal{H}$ is basically a 2D grid were each value h_{ij} of the grid represents the height at one cell (i, j) of the gird, we can then use the points of the grid to obtain the elevation on a continuos domain using interpolation. Because we already have a defined grid, we can use it to implement our sampling. The main idea is to make use of the 2D grid and extend it to a 3D grid by setting a number of vertical samples (which will be a user-difine parameter) and then sample one point for each element of the grid, as long as the point lies inside the terrain (the 3D grid will represent the bounding box of the terrain, so not every voxel will be inside it).

Now that we have a way to uniformly sample points inside the model, we can extend it and break the regularity a little bit by shifting each point by a certain quantity. Considering each point to initially be sampled at the center of its grid cell, we are going to shift each point by *strictly* less than half the cell size in a random direction, this way be will obtain a sampling in between complete random sampling (white noise) and a completly regular sampling, where we guarantee each point to not be to close or to far to another. Specifically, from each grid cell center c_{ijk} we create our sample c'_{ijk} using the following equation:

$$c'_{ijk} = c_{ijk} + \begin{bmatrix} r_x s_x \\ r_y s_y \\ r_z s_z \end{bmatrix} \tag{4.5}$$

Where s_x, s_y and s_z are the cell sizes on each direction (we can have rectangular cells) and r_x, r_y and r_z are three different scalars sampled uniformly at random such as $-\frac{1}{2} < -m \leq r_k \leq m < \frac{1}{2}$, where m denotes the maximum ammount of shifting allowed and $k \in \{x, y, z\}$. If m was $\frac{1}{2}$ then samples will be allowed to lie on the boundary of the grid cell, which could result in two samples being at the same place, that is why we set m to be *strictly* smaller than $\frac{1}{2}$.



(a) Voronoi cells of the decomposition

(b) Sample points with the base model as reference

Figure 4.3: Result of the voronoi decomposition without cutting the boundary cells vs. the samples used for the decomposition with the base model as reference

Moreover, we can bound the distance between two arbitrary points in terms of m and the cell sizes s_k. Suppose s_x is the smaller size, then the minimum posible distance between two samples $c_{i,j,k}$ and $c'_{i+1,j,k}$ appears when $c'_{i,j,k} = (x + ms_x, y + \alpha s_y, z + \beta s_z)$ and $c'_{i+1,j,k} = (x + s_x - ms_x, y + \alpha s_y, z + \beta s_z)$, so the distance is:

$$d(c'_{ijk}, c'_{i+1,j,k}) = ||c'_{i+1,jk} - c'_{ijk}|| = ||(s_x - 2ms_x, 0, 0)|| = s_x(1 - 2m) \tag{4.6}$$

Following a similar logic, the maximum distance between $c'_{i,j,k}$ and its closest sample $c'_{i+1,j,k}$, will be when $c'_{i,j,k} = (x - ms_x, y - ms_y, z - ms_z)$ and $c'_{i+1,j,k} = (x + s_x + ms_x, y + ms_y, z + ms_z)$ and the distance would be:

$$d(c'_{ijk}, c'_{i+1,j,k}) = ||c'_{i+1,jk} - c'_{ijk}|| = ||(s_x + 2ms_x, 2ms_y, 2ms_z)|| \tag{4.7}$$

But keep in mind that this maximum distance will only be reached when $i = 0$, as in any other case c'_{ijk} would be closer to $c'_{i-1,j,k}$ than $c'_{i+1,j,k}$ (defining c'_{ijk} and $c_{i+1,j,k}$ as we did). An analogous argument can be repeated with s_y or s_z as the smaller size.

To sum up, the method we have chosen to generate samples bounds the minum and maximum distances between a sample and its closest pair, so it combines some regularity that comes with using a grid with some randomness that breaks the monotony and creates more natural shapes.

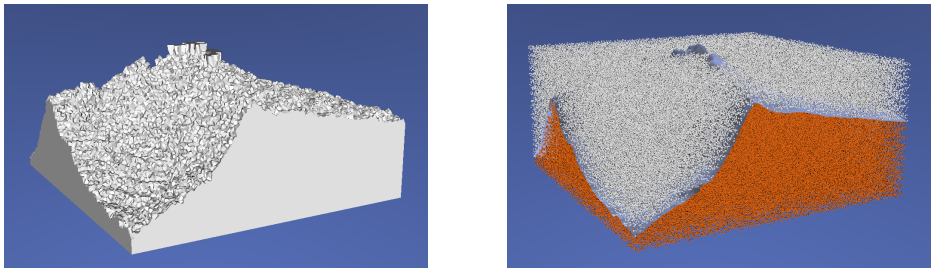
INSERT A FIGURE WITH LONGEST AND SHORTEST DISTANCE CASES???

4.2.3 Wall Construction

The last consideration we have to make to create the voronoi cells is about what do we do with cells on the boundary of the terrain. The method proposed on [26] relied on boolean operations between the model and the set of voronoi cells to cut the boundary cells, but in this project we have tried multiple methods to appropiately create the boundary cells without relying on expensive boolean operations between models.

Sampling out of Bounds

If we do nothing more than sampling points and computing its voronoi cells, some cells will extend until the bounding box of the terrain and the resulting decomposition will result on just a giant rectangular prism made of internal voronoi cells. The problem, as we mentioned earlier, is that the boundary voronoi



(a) Voronoi cells of the decomposition

(b) Sample points with the base model as reference. White particles are out of bounds

Figure 4.4: Result of the voronoi decomposition when allowing sampling out of bounds

cells are not cut, but we can trivially try to solve this problem by adding more cells outside that boundary, so the cells on the boundary of the terrain stop being the boundary and thus are cut by other cells. The samples outside of the terrain generate *air* cells and thus they are not rendered, so the only rendered cells are the internal ones, which are properly cut.

The problem of generating a giant rectangular prism is solved using this method, but the surface of the terrain will have a really irregular shape, as cells are arbitrarily cut. Despite that we can use this idea as a base and then try to improvit so the surface of the model resembles more closely the original.

Using Additional Samples to Generate Cutting Planes

As explained on section 4.2.1, every pair of samples define a plane P_{ij} with all points that are equidistant from both samples. Additionally, we know that region R_i will be cut by all planes P_ij such as $i \neq j$, so we can use this concept in order to generate new cutting planes using more samples for the voronoi cell generation algorithm.

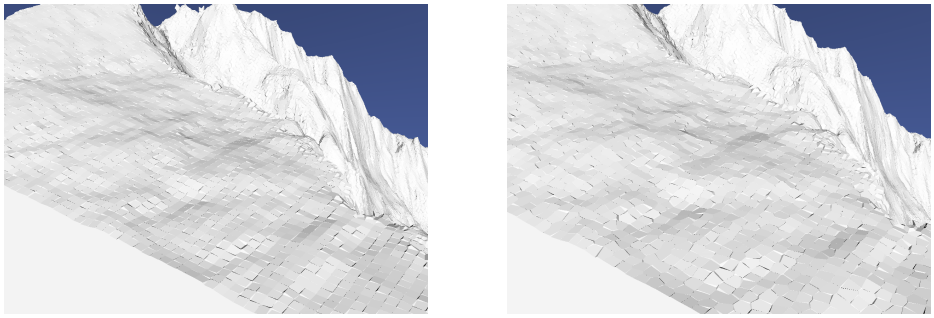
As we mentioned on the preliminaries 2.2.1, we can sample normal vectors from DEMs, which means we can extract a plane from every point of the terrain surface. If we use this concept, we can create a plane for every cell, bound it using the size of the cell and place it on the height of the given cell (so it lies on the surface) to obtain a decomposition of the terrain surface using quads. Moreover we can use the idea of sampling points outside the boundary to cut the cells on the surface of the terrain to create an approximation of this quad tesselation.

For every grid cell (i, j) of the heightfield $\mathcal{H}$ we can extract its height at the center of the cell h_{ij} and also its normal $n_{i,j}$ to create a plane π_{ij} going thorugh point (x_{ij}, y_{ij}, h_{ij}) with normal n_{ij}. With this we can create two samples p_{ij} and q_{ij} such as:

$$p_{ij} = \begin{bmatrix} x_{ij} \\ y_{ij} \\ h_{ij} \end{bmatrix} - kn_{ij} \tag{4.8}$$

$$q_{ij} = \begin{bmatrix} x_{ij} \\ y_{ij} \\ h_{ij} \end{bmatrix} + kn_{ij} \tag{4.9}$$

So that p_{ij} is inside the terrain, q_{ij} outside and both points are at the same distance k from the plane,



(a) Results sampling at cell center (b) Results adding some jittering

Figure 4.5: Wall results sampling at the center of every cell vs. adding some jittering

which implies that the plane P_{pq} generated by those two points during the voronoi cell computation will be exactly the plane π_{ij}. Applying this concept to every grid cell will produce a better surface than the one generated by just sampling out of bounds, as we cut the boundary cells following the shape of the terrain. The only thing that we should take into account is to choose an appropiate value of k.

Additionally, because we can interpolate the values of the grid to obtain the height $h(x, y)$ and normal $n(x, y)$ for any point inside the grid bounds, we can apply some jittering to break the regularity and create differently shaped cells that follow the surface. Given $x'_{ij} = x_{ij} + ms_x$ and $y'_{ij} = y_{ij} + ms_y$ (where m and s have the same definition as the one we used in section 4.2.2), we define the shifted points p'_{ij} and q'_{ij} as:

$$p'_{ij} = \begin{bmatrix} x'_{ij} \\ y'_{ij} \\ h(x'_{ij}, y'_{ij}) \end{bmatrix} - kn(x_{ij}, y_{ij}) \tag{4.10}$$

$$q'_{ij} = \begin{bmatrix} x'_{ij} \\ y'_{ij} \\ h(x'_{ij}, y'_{ij}) \end{bmatrix} + kn(x_{ij}, y_{ij}) \tag{4.11}$$

The idea of this method to generate an appropiate boundary is to mix it with the general sampling method that we described, so we will have all samples form the general method and an additional sample per cell that will represent the surface. This mix of samples initially supposes a problem, as the minimum distance bound that we showed in section 4.2.2 might not hold between points sampled from different methods. When we tried to mix bot sampling methods without further consideration, some bumps appeared (as seen in figure 4.6) on the surface of the terrain due to a mix of samples close to the surface. To solve this we limited the allowed height for samples using the initial sampling method so that the only samples close to the surface were the ones generated with their corresponding pair.

Finally, there was another problem when using this method to create the walls.

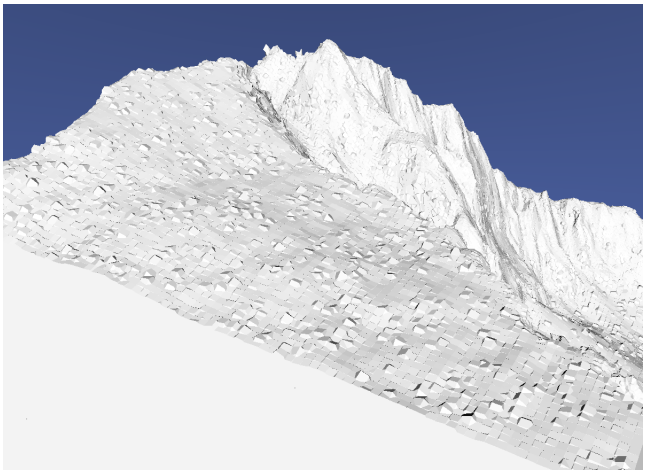
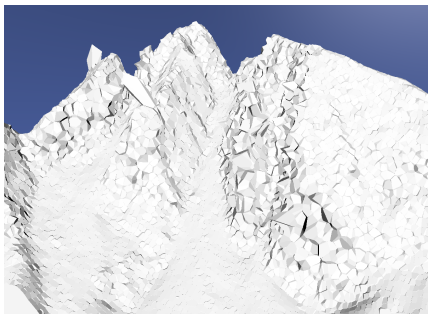
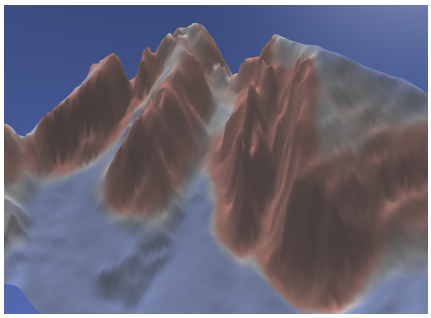


Figure 4.6: Bumps appearing due to an unappropiate mix of the two sampling methods



(a) Base results of our wall generating method



(b) Base terrain painted with a Cool-Warm color gradient. The warmer the color the higher the rugosity of that point

Figure 4.7: Results of the wall generation process vs. a color map over the base terrain showing the rugosity at each point

4.3 Link Generation

4.3.1 Link Adjacency Information

4.3.2 Link Parameters

4.4 Multi-Resolution Extension

Chapter 5

Erosion

5.1 Initial Method

5.2 Multi-resolution

5.3 Physical Model

5.4 Extensions

Chapter 6

Implementation

Chapter 7

Results

Chapter 8

Conclusions

Summary of the previous results, final conclusions and closing thoughts

8.1 Future Work

8.2 Conclusions

Bibliography

[1] Diego Mateos et al. "Simulation of Mechanical Weathering for Modeling Rocky Terrains ". In: *Spanish Computer Graphics Conference (CEIG)*. Ed. by Julio Marco and Gustavo Patow. The Eurographics Association, 2024. ISBN: 978-3-03868-261-5. DOI: 10.2312/ceig.20241139.

[2] O. Argudo et al. "Terrain descriptors for landscape synthesis, analysis and simulation". In: *Computer Graphics Forum* 44.2 (2025). DOI: https://doi.org/10.1111/cgf.70080.

[3] ChrisH. Rycroft. *VORO++*. Feb. 2009. DOI: 10.11578/dc.20210416.28. URL: https://www.osti.gov/biblio/code-54674.

[4] Eric Galin et al. "A review of digital terrain modeling". In: *Computer Graphics Forum*. Vol. 38. 2. Wiley Online Library. 2019, pp. 553–577.

[5] B. Benes and R. Forsbach. "Layered data representation for visual simulation of terrain erosion". In: *Proceedings Spring Conference on Computer Graphics*. 2001, pp. 80–86. DOI: 10.1109/SCCG.2001.945341.

[6] Guillaume Cordonnier et al. "Authoring landscapes by combining ecosystem and terrain erosion simulation". In: *ACM Trans. Graph.* 36.4 (July 2017). ISSN: 0730-0301. DOI: 10.1145/3072959.3073667. URL: https://doi.org/10.1145/3072959.3073667.

[7] Dennis Wingender and Daniel Balzani. "Simulation of crack propagation through voxel-based, heterogeneous structures based on eigenerosion and finite cells". In: *Computational Mechanics* 70.2 (2022), pp. 385–406.

[8] Giuseppe Turini, Fabio Ganovelli, and Claudio Montani. "Simulating Drilling on Tetrahedral Meshes." In: *Eurographics (Short Presentations)*. 2006, pp. 127–131.

[9] Matias Muller. *100 X simulation speedup with mesh embedding*. 2022. URL: https://matthias-research.github.io/pages/tenMinutePhysics/12-softBodySkinning.pdf.

[10] Alan Norton et al. "Animation of fracture by physical modeling". In: *Vis. Comput.* 7.4 (July 1991), 210–219. ISSN: 0178-2789. DOI: 10.1007/BF01900837. URL: https://doi.org/10.1007/BF01900837.

[11] Demetri Terzopoulos and Kurt Fleischer. "Modeling inelastic deformation: viscolelasticity, plasticity, fracture". In: *Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '88. New York, NY, USA: Association for Computing Machinery, 1988, 269–278. ISBN: 0897912756. DOI: 10.1145/54852.378522. URL: https://doi.org/10.1145/54852.378522.

[12] Joshua A. Levine et al. "A Peridynamic Perspective on Spring-Mass Fracture". In: *Eurographics/ACM SIGGRAPH Symposium on Computer Animation*. Ed. by Vladlen Koltun and Eftychios Sifakis. The Eurographics Association, 2014. ISBN: 978-3-905674-61-3. DOI: 10.2312/sca.20141122.

[13] Saty Raghavachary. "Fracture generation on polygonal meshes using Voronoi polygons". In: *ACM SIGGRAPH 2002 Conference Abstracts and Applications*. SIGGRAPH '02. San Antonio, Texas: Association for Computing Machinery, 2002, p. 187. ISBN: 1581135254. DOI: 10.1145/1242073.1242200. URL: https://doi.org/10.1145/1242073.1242200.

[14] Adrien Peytavie et al. "DeadWood: Including Disturbance and Decay in the Depiction of Digital Nature". In: *ACM Trans. Graph.* 43.2 (Feb. 2024). ISSN: 0730-0301. DOI: 10.1145/3641816. URL: https://doi.org/10.1145/3641816.

[15] Bosheng Li et al. "Stressful Tree Modeling: Breaking Branches with Strands". In: *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*. SIGGRAPH Conference Papers '25. New York, NY, USA: Association for Computing Machinery, 2025. ISBN: 9798400715402. DOI: 10.1145/3721238.3730745. URL: https://doi.org/10.1145/3721238.3730745.

[16] ZHANYU YANG and NIKOLAS A. SCHWARZ. 2026. URL: https://jango6324.github.io/woodstock/.

[17] T. Ito et al. "Modeling rocky scenery taking into account joints". In: *Proceedings Computer Graphics International 2003*. 2003, pp. 244–247. DOI: 10.1109/CGI.2003.1214475.

[18] Zhanyu Yang et al. "Arenite: A Physics-based Sandstone Simulator". In: *ACM Trans. Graph.* 44.4 (July 2025). ISSN: 0730-0301. DOI: 10.1145/3731201. URL: https://doi.org/10.1145/3731201.

[19] Axel Paris et al. "Terrain Amplification with Implicit 3D Features". In: *ACM Transactions on Graphics* 38.5 (2019). DOI: 10.1145/3342765. URL: https://hal.science/hal-02273097.

[20] Axel Paris et al. "Modeling Rocky Scenery using Implicit Blocks". In: *The Visual Computer* 36.10 (2020), pp. 2251–2261. DOI: 10.1007/s00371-020-01905-6. URL: https://hal.science/hal-02926218.

[21] Axel Paris et al. "Synthesizing Geologically Coherent Cave Networks". In: *Computer Graphics Forum* (2021). DOI: 10.1111/cgf.14420. URL: https://hal.science/hal-03331697.

[22] Guillaume Cordonnier et al. "Sculpting Mountains: Interactive Terrain Modeling Based on Subsurface Geology". In: *IEEE Transactions on Visualization and Computer Graphics* 24.5 (May 2018), pp. 1756–1769. DOI: 10.1109/TVCG.2017.2689022. URL: https://hal.science/hal-01517343.

[23] Éric Guérin et al. "Interactive example-based terrain authoring with conditional generative adversarial networks". In: *ACM Trans. Graph.* 36.6 (Nov. 2017). ISSN: 0730-0301. DOI: 10.1145/3130800.3130804. URL: https://doi.org/10.1145/3130800.3130804.

[24] C. Grenier et al. "Real-time Terrain Enhancement with Controlled Procedural Patterns". In: *Computer Graphics Forum* 43.1 (2024), e14992. DOI: https://doi.org/10.1111/cgf.14992. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.14992. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.14992.

[25] O. Argudo, A. Chica, and C. Andujar. "Terrain Super-resolution through Aerial Imagery and Fully Convolutional Networks". In: *Computer Graphics Forum* 37.2 (2018), pp. 101–110. DOI: https://doi.org/10.1111/cgf.13345. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.13345. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.13345.

[26] Diego Mateos Arlanzón. "Simulation of mechanical weathering for modeling rocky terrains". Projecte Final de Màster Oficial. UPC, Facultat d'Informàtica de Barcelona, Departament de Ciències de la Computació, Oct. 2023. URL: https://hdl.handle.net/2117/403778.

[27] Emanuel A. Lazar, Jiayin Lu, and Chris H. Rycroft. "Voronoi cell analysis: The shapes of particle systems". In: *American Journal of Physics* 90.6 (June 2022), pp. 469–480. ISSN: 0002-9505. DOI: 10.1119/5.0087591. eprint: https://pubs.aip.org/aapt/ajp/article-pdf/90/6/469/20100696/469_1_5.0087591.pdf. URL: https://doi.org/10.1119/5.0087591.

[28] John P. Wilson. "Digital terrain modeling". In: *Geomorphology* 137.1 (2012). Geospatial Technologies and Geomorphological Mapping Proceedings of the 41st Annual Binghamton Geomorphology Symposium, pp. 107–121. ISSN: 0169-555X. DOI: https://doi.org/10.1016/j.geomorph.2011.03.012. URL: https://www.sciencedirect.com/science/article/pii/S0169555X11001449.

[29] Farès Belhadj. "Terrain modeling: a constrained fractal model". In: *Proceedings of the 5th International Conference on Computer Graphics, Virtual Reality, Visualisation and Interaction in Africa*. AFRIGRAPH '07. Grahamstown, South Africa: Association for Computing Machinery, 2007, 197–204. ISBN: 9781595939067. DOI: 10.1145/1294685.1294717. URL: https://doi.org/10.1145/1294685.1294717.

[30] Mattia Natali et al. "Modeling Terrains and Subsurface Geology." In: *Eurographics (State of the Art Reports)*. 2013, pp. 155–173.

[31] Demetri Terzopoulos and Kurt Fleischer. "Modeling inelastic deformation: viscolelasticity, plasticity, fracture". In: *SIGGRAPH Comput. Graph.* 22.4 (June 1988), 269–278. ISSN: 0097-8930. DOI: 10.1145/378456.378522. URL: https://doi.org/10.1145/378456.378522.